

P1 · FUNÇÕES (envolvem o valor)

<code>int("123") → 123</code>	texto → inteiro; mata zero à esq. (<code>int("02") → 2</code>)
<code>float("12.5") → 12.5</code>	texto → decimal (ponto, NUNCA vírgula)
<code>str(42) → "42"</code>	número → texto
<code>input("msg")</code>	lê do usuário; retorna SEMPRE str
<code>print(x, y)</code>	mostra; vírgula põe espaço entre args
<code>len(x)</code>	tamanho: lista, string, df (nº de linhas)
<code>round(x, 2)</code>	arredonda em 2 casas
<code>sum(l) / max(l) / min(l)</code>	soma / maior / menor de lista
<code>range(5)</code>	0,1,2,3,4 (pra <code>for i in range(n)</code>)
<code>type(x)</code>	debug: qual o tipo disso?

TEU ERRO Nº1 (5x): função RETORNA, não altera. `int(x)` solto joga fora. SEMPRE `x = int(x)`. Depois de TODO `input/split`: "converto pra quê? atribui?". Comparar/contar com `str = TypeError` ou `False` eterno (`"1" == 1` é `False`).

△ Preço em `input = float()`, nunca `int()` (centavos quebram `int("10.5")`).

P2 · STRINGS (.depois do valor)

<code>s.split(" ")</code>	corta → LISTA de str (converter números!)
<code>s.upper()</code> / <code>s.lower()</code>	MAIÚSCULA / minúscula (atribuir!)
<code>s.strip()</code>	tira espaços das pontas
<code>s.isdigit()</code>	<code>True</code> só se TUDO é dígito ("12.3", "-5", "12 " → <code>False</code>)
<code>s.replace("a","b")</code>	troca todas as ocorrências
<code>s[0:4]</code>	fatia: posições 0,1,2,3 (fim NÃO entra; conta do 0)
<code>"x" in s</code> / <code>x in lista</code>	contém?
<code>a, b, c = s.split("/")</code>	desempacota: 1 nome por pedaço (nº igual!)
<code>partes[0]</code>	caixinha 0 da lista (= coluna A do Texto p/ Colunas)

△ Dois separadores num `split` NÃO existe: `split` em ESTÁGIOS (espaço primeiro separa os mundos, depois / abre a data). Campo opcional: só acessar `partes[1]` DENTRO de `if len(partes)==2:`.

P3 · F-STRING (montar texto)

<code>f"0i {nome}"</code>	f liga o modo; {} injeta variável
<code>f"{x:.2f}" / f"{x:.1f}"</code>	2 casas / 1 casa decimal

UMA string contínua. Sem +, sem {} fora do f". "Paciente" x (justaposição) = `SyntaxError`. Escreve a frase literal primeiro, depois envolve as variáveis com chaves. 3.5999... não é bug: float binário, formata com `:.2f` e segue.

P4 · CONDIÇÕES

<code>==</code> <code>!=</code> <code>>=</code> <code><=</code>	comparar VALOR (nunca <code>is</code> / <code>is not</code>)
<code>x not in ["D","N"]</code>	"nem D nem N" (o idioma certo)
<code>x != "D" and x != "N"</code>	equivalente verboso (com <code>!=</code> o conector é AND)
<code>x >= A and x <= B</code>	"entre A e B (inclusive)": $\geq$ menor E $\leq$ maior

x != "D" or "N" é SEMPRE True ("N" sozinho é `truthy`). Cada lado do `and/or` = comparação completa. Enunciado: OU = `or`, E = `and`, literal. "Até X" INCLUI X (`<=`): borda errada = rubrica inteira (sem parcial).

P5 · LAÇOS: FOR vs WHILE

<code>for t in range(3):</code>	# nº de voltas CONHECIDO antes
<code>for item in lista:</code>	# visita cada item da coleção
<code>while saldo >= 4:</code>	# repete ENQUANTO condição valer
<code>saldo -= 4</code>	# algo muda a cada volta

△ Justificativa pronta: quantidade fixa antes do `laço = for` · "enquanto tiver/der" = `while` · "R\$40 dá 10 tentativas" é isca: `saldo` é variável, parada é condição. Bloco fecha DESIDENTANDO (`return` é de função, não fecha `laço`). Dentro do `laço` o item é `i`, nunca a lista (`split` é na fruta, não na cesta).

P6 · ESQUELETO: VALIDAÇÃO + FAIXAS

```
d = input("Dist: ")
uf = input("UF: ").upper()
if not d.isdigit() or uf not in ["SP","PR"]:
    print("ERRO DE ENTRADA") # valida ANTES do int()
else:
    d = int(d)
    if d <= 2000: # "até X" inclui X
        taxa = 5.0
    elif d <= 8000: # elif = faixa seguinte
        taxa = d/1000 * 1.2 # 1.2 PONTO! (1,2 = tup
    else:
        taxa = 25.0
    if uf == "PR":
        taxa *= 2 # dobro vale p/ fixo tb
    print(f"R$ {taxa:.2f}")
```

P7 · ESQUELETO: ACUMULADOR (Q5)

```
cont = 0; total = 0 # zera FORA
for item in lista: # item = 1 string
    partes = item.split("|")
    valor = float(partes[2]) # converte E atribui
    cont += 1
    total += valor
    print(f"... {partes[0]} ...") # 1 por item
print(f"{cont} x - Total: R$ {total:.2f}") # FORA
```

P8 · ESQUELETO: DATA C/ CAMPO OPCIONAL

```
meses = ['janeiro','fevereiro','março','abril',
'maio','junho','julho','agosto','setembro',
'outubro','novembro','dezembro']
for reg in datas:
    partes = reg.split(" ") # grosso 1º
    dia, mes, ano = partes[0].split("/")
    txt = f"{int(dia)} de {meses[int(mes)-1]} de {ano}"
    if len(partes) == 2: # guarda!
        h, m = partes[1].split(":")
        h = int(h); m = int(m)
        txt += " à 1 hora" if h==1 else f" às {h} hora
        if m == 1: txt += " e 1 minuto"
        elif m > 1: txt += f" e {m} minutos"
    print(txt) # m==0: omite
# data junta: ano=s[0:4] mes=s[4:6] dia=s[6:8]
```

PANDAS - sempre `import pandas as pd` - arquivos na MESMA pasta do notebook: caminho = só o nome

D1 - CRIAR / LER	
<code>df = pd.read_excel("a.xlsx")</code>	planilha Excel
<code>pd.read_excel("a.xlsx", index_col=0)</code>	1ª coluna vira índice
<code>pd.read_csv("a.csv")</code>	CSV padrão (vírgula separa, ponto decimal)
<code>pd.read_csv("a.csv", sep=";", decimal=",")</code>	CSV BRASILEIRO: sem isso, nº vira texto
<code>df = pd.DataFrame({"time": ["Palmeiras", "Flamengo"], "pontos": [68, 68]}) # dict: chave=coluna</code>	
⚠ Teste pós-leitura: <code>df.dtypes</code> → valor tem que ser <code>float64/int64</code>; <code>object</code> = virou texto, faltou <code>decimal=", "</code>. Confere o ARQUIVO certo da questão (MrPacoca ≠ pedidos).	
D2 - EXPLORAR	
<code>df.head()</code> / <code>df.tail()</code>	5 primeiras / últimas linhas (DF INTEIRO, todas col.)
<code>df.shape</code>	(linhas, colunas), sem parênteses
<code>list(df.columns)</code>	nomes EXATOS (acento conta; errado = <code>KeyError</code>)
<code>df.dtypes</code> / <code>df.info()</code>	tipos / resumo
<code>df["col"].unique()</code>	QUAIS valores existem ("Pote", não "Potes")
D3 - FILTRAR / ALTERAR / 1 LINHA	
<code>df["col"]</code>	uma coluna (Series)
<code>df["col"] == "SP"</code>	MÁSCARA: True/False por linha
<code>df[df["col"] == "SP"]</code>	só as linhas True (tabela filtrada)
<code>df.loc[mask, "col"] = v</code>	REGRA DE OURO p/ trocar valor filtrado
<code>df.loc[mask, "col"] += 0.04</code>	somar só nos filtrados
<code>df["nova"] = df["a"] * df["b"]</code>	criar coluna (vetorizado, sem laço)
<code>linha = df[df["id"]==x].iloc[0]</code> <code>x in df["col"].values</code>	UMA linha como ficha → <code>linha["col"]</code> existe? SEM <code>.values</code> checa o ÍNDICE
<code>df["col"].isin(["A","B"])</code> <code>df["col"].astype(str)</code>	máscara: está na lista? converter coluna (nº ↔ texto p/ comparar)
<code>df["col"].str[0:3]</code> / <code>.str.upper()</code>	<code>.str</code> = métodos de string na coluna inteira (fatia SÓ em colchetes)
TABELA × LINHA × ITEM (3 crashes): "truth value of a Series is ambiguous" = coluna inteira dentro de if. Depois do <code>.iloc[0]</code> , TUDO vem de <code>linha[...]</code> ; dentro de loop, de <code>i</code> . <code>df = todo mundo</code> · <code>linha</code> = um registro.	
DIREÇÃO: reajuste/aumento = <code>+=</code> · desconto = <code>-=</code> · diferença pode coincidir e esconder o erro: confere <code>head()</code> após a troca. NUNCA <code>df[df[...]==x]["col"] = v</code> (cópia). FutureWarning/Chained: IGNORAR (enunciado manda).	

D4 - AGREGAR / CONTAR	
<code>df["col"].sum()</code> · <code>.mean()</code> · <code>.min()</code> · <code>.max()</code>	soma, média, mín, máx
<code>df["col"].agg(["min", "mean", "max"]).round(2)</code>	vários stats em UMA operação (exigência típica)
<code>df["col"].value_counts()</code>	contagem POR categoria (NÃO fazer loop c/ contadores!)
<code>df["col"].nunique()</code>	QUANTAS categorias distintas
<code>mask.sum()</code>	conta True (True=1). <code>.count()</code> = não-nulos, NÃO use
<code>serie.sort_values(ascending=False)</code>	decrescente; <code>.head(5)</code> = top 5
D5 - ANÁLISE (Excel → pandas)	
<code>df.groupby("g")["v"].mean().sort_values(ascending=False)</code>	Tabela Dinâmica de 1 medida = ranking por grupo
<code>df.groupby("g")["v"].agg(["mean", "max"])</code>	vários stats por grupo
<code>pd.merge(a, b, on=["k1", "k2"])</code>	PROCV por N chaves, chave faltando MULTIPLICA linhas (teste: shape!)
<code>df.pivot_table(index="Estado", columns="Produto", values="Total", aggfunc="mean").reset_index()</code>	Tabela Dinâmica: <code>index=Linhas</code> , <code>columns=Colunas</code> , <code>values=Valores</code> , <code>aggfunc=Resumir</code> por <code>df.columns.name=None</code> limpa rótulo
<code>(df["m"] < 0).replace({True:"prejuízo", False:"lucro"})</code>	classificar sem if
<code>np.where(cond, "sim", "não")</code>	idem (import numpy as np)
⚠ TRIAGEM: dataset grande + média/ranking/cruzar = pandas SEM <code>for</code>. Lista de strings item a item = <code>for</code> + <code>split</code> (lado PYTHON). Ranking liderado por grupo de amostra MÍNIMA = liderança frágil: olha o <code>value_counts()</code> antes de confiar.	

D6 - ESQUELETO: BUSCA ID + ÁRVORE	
<pre>pid = int(input("id: ")) # int: coluna é nº! if pid not in df["id"].values: print("Paciente não encontrado") else: linha = df[df["id"]==pid].iloc[0] imc = linha["peso"]/linha["altura"]**2 if linha["sist"] >= 140 or linha["diast"] >= 90: r = "consulta imediata" # regra 1 GANHA elif imc >= 30 and linha["fum"]==sim: r = "risco alto" # ordem elif = prior elif imc >= 30 or linha["idade"] >= 60: r = "risco moderado" # OU do enunciado = else: r = "risco baixo" print(f"Paciente {pid} (IMC: {imc:.1f}) – Risco: {r}") # travessão – : COPIA do enunciado</pre>	
D7 - ESQUELETO: REAJUSTE + DIFERENÇA	
<pre>total_pre = df["Total"].sum() # guarda ANTES df["Preço Unitário"] = df["Total"]/df["Quantidade"] df.loc[df["Mês"]==mes, "Preço Unitário"] += reajuste df["Total"] = df["Preço Unitário"]*df["Quantidade"] print(f"R\$ {df['Total'].sum()-total_pre:.2f}") # usa as VARIÁVEIS dadas, nunca os nº do exemplo</pre>	
⚡ PROTOCOLO (10 passos)	
0. Q0 primeiro	nome, turma AE_, código; não apagar os #
1. Triagem 60s	cada questão: conceito / ler pandas / if+validação / DataFrame / parse em laço / análise
2. Ordem	Q1→Q2 → Q5 (3pts) → Q3 → Q4. Travou 10min: pula e volta
3. Antes do código	<code>df.columns + head()</code> : nomes e valores REAIS
4. Consertos em LOTE	achou 3 problemas? conserta os 3, DEPOIS roda
5. Teste de fechamento	exemplo do enunciado + borda exata de cada faixa + caso de ERRO + UMA entrada de CADA categoria/estado
6. Warnings	SyntaxWarning DIZ o conserto; FutureWarning: ignora
7. Saida completa	pediu 3 valores? imprime os 3 (faltar 1 = rubrica zero)
8. Formato exato	copia símbolo do enunciado · <code>:.2f</code> · linha em branco do modelo
9. Run All	Restart & Run All sem erro (prosa em célula de código = TODA linha com #)
10. Varredura final	NENHUMA resposta em branco · Q0 preenchida